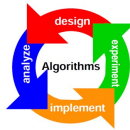


Estrategias algorítmicas

José A. Rodríguez Melquiades

21 de junio de 2026

Algoritmos dividir para vencer



- 1 Dividir para vencer (conquistar)
- 2 Aplicaciones de algoritmos basados en Dividir para vencer
- 3 Problema del par mas cercano
- 4 Valor máximo y mínimo de un arreglo

Dividir para vencer (conquistar)

- Proceso:

Dividir para vencer (conquistar)

- **Proceso:**

- ▶ **Dividir:** Un problema se divide en varios subproblemas del mismo tipo. Preferentemente de igual tamaño.

Dividir para vencer (conquistar)

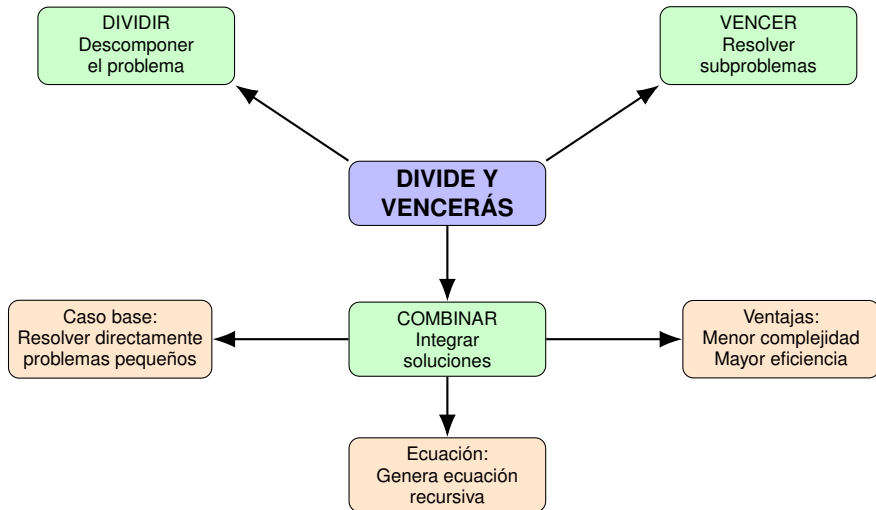
- **Proceso:**

- ▶ **Dividir:** Un problema se divide en varios subproblemas del mismo tipo. Preferentemente de igual tamaño.
- ▶ **Conquistar** Los subproblemas se solucionan generalmente en forma recursiva.

Dividir para vencer (conquistar)

- **Proceso:**

- ▶ **Dividir:** Un problema se divide en varios subproblemas del mismo tipo. Preferentemente de igual tamaño.
- ▶ **Conquistar** Los subproblemas se solucionan generalmente en forma recursiva.
- ▶ **Combinar:** Las soluciones de los subproblemas se combinan para obtener la solución final del problema.



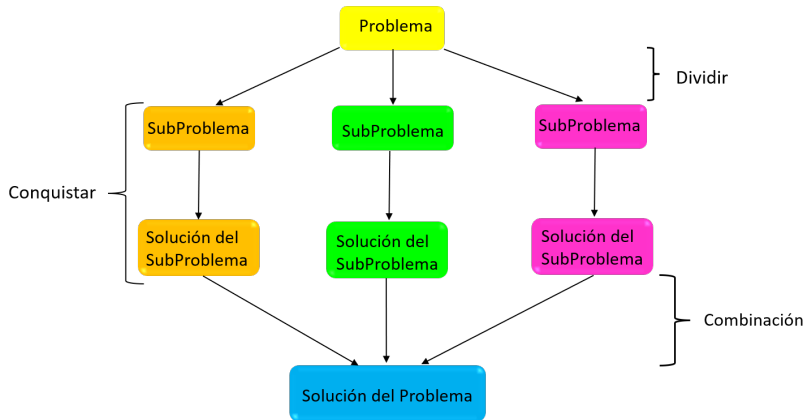


Figura 1: División del problema en subproblemas y su solución.

- Ejemplo:

- Ejemplo:

- ▶ Sea una sucesión de n números dados como:

$$\{a_1, a_2, \dots, a_{n-2}, a_{n-1}, a_n\}$$

Se desea sumar los números.

- Ejemplo:

- ▶ Sea una sucesión de n números dados como:

$$\{a_1, a_2, \dots, a_{n-2}, a_{n-1}, a_n\}$$

Se desea sumar los números.

- ★ Dividir la sucesión de números en otras dos subsucesiones:

$$\{a_1 + \dots + a_{\lfloor n/2 \rfloor - 1}\} + \{a_{\lfloor n/2 \rfloor} + a_{\lfloor n/2 \rfloor + 1} + \dots + a_n\}$$

- Ejemplo:

- ▶ Sea una sucesión de n números dados como:

$$\{a_1, a_2, \dots, a_{n-2}, a_{n-1}, a_n\}$$

Se desea sumar los números.

- ★ Dividir la sucesión de números en otros dos subsucesiones:

$$\{a_1 + \dots + a_{\lfloor n/2 \rfloor - 1}\} + \{a_{\lfloor n/2 \rfloor} + a_{\lfloor n/2 \rfloor + 1} + \dots + a_n\}$$

- ★ ¿es eficiente para calcular la suma de los n números ?; ¿es mejor que la estrategia por fuerza bruta?.

● Ejemplo:

- ▶ Sea una sucesión de n números dados como:

$$\{a_1, a_2, \dots, a_{n-2}, a_{n-1}, a_n\}$$

Se desea sumar los números.

- ★ **Dividir** la sucesión de números en otros dos subsucesiones:

$$\{a_1 + \dots + a_{\lfloor n/2 \rfloor - 1}\} + \{a_{\lfloor n/2 \rfloor} + a_{\lfloor n/2 \rfloor + 1} + \dots + a_n\}$$

- ★ ¿es eficiente para calcular la suma de los n números ?; ¿es mejor que la estrategia por fuerza bruta?.
- ★ **Rpta.:** No todos los algoritmos de divide y vencerás son necesariamente más eficientes que incluso una solución de fuerza bruta; pues el tiempo empleado por dividir y venceras puede ser mayor que otra estrategia pueda resolver.

- **Ejemplo:**

- ▶ Sea una sucesión de n números dados como:

$$\{a_1, a_2, \dots, a_{n-2}, a_{n-1}, a_n\}$$

Se desea sumar los números.

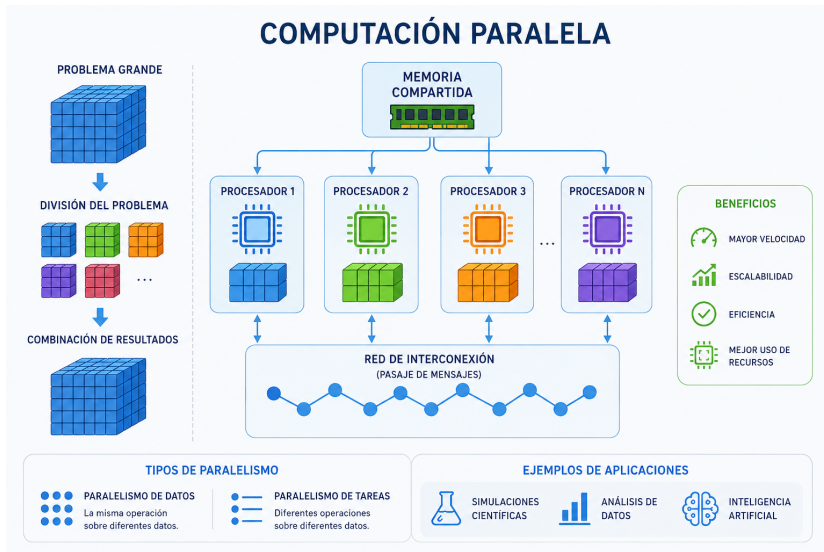
- ★ **Dividir** la sucesión de números en otras dos subsucesiones:

$$\{a_1 + \dots + a_{\lfloor n/2 \rfloor - 1}\} + \{a_{\lfloor n/2 \rfloor} + a_{\lfloor n/2 \rfloor + 1} + \dots + a_n\}$$

- ★ ¿es eficiente para calcular la suma de los n números ?; ¿es mejor que la estrategia por fuerza bruta?.
- ★ **Rpta.:** No todos los algoritmos de divide y vencerás son necesariamente más eficientes que incluso una solución de fuerza bruta; pues el tiempo empleado por dividir y venceras puede ser mayor que otra estrategia pueda resolver.

- **Observación:** La estrategia divide y vencerás produce algunos de los algoritmos más importantes y eficientes de la ciencia de la computación.

- **Observación 1:** La estrategia DV puede ser usado en:



 EN COMPUTACIÓN PARALELA

Las arquitecturas paralelas se clasifican según la forma en que los procesadores se conectan y se comunican entre sí.

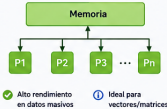
1. SISD – Secuencial

Un solo procesador ejecuta las instrucciones de manera secuencial.



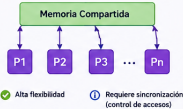
2. SIMD – Paralelismo de Datos

Múltiples procesadores realizan la misma instrucción sobre diferentes datos.



3. MIMD – Memoria Compartida

Cada procesador puede ejecutar diferentes instrucciones accediendo a una memoria común.



4. MIMD – Memoria Distribuida

Cada procesador tiene su propia memoria local y se comunica mediante una red.



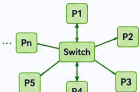
TOPOLOGÍAS DE INTERCONEXIÓN (para memoria distribuida)

Bus



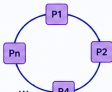
- ✓ Simple y económico
- ✗ Escalabilidad limitada

Estrella



- ✓ Fácil de administrar
- ✗ El switch puede ser cuello de botella

Anillo



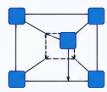
- ✓ Comunicación ordenada
- ✗ Falla en un nodo afecta al anillo

Malla 2D



- ✓ Alta escalabilidad
- ⓘ Mayor complejidad de conexiones

Hipercubo



- ✓ Baja latencia
- ⓘ Complejo de implementar



La elección de la organización y topología depende del problema, el hardware disponible y los objetivos de rendimiento.

- **Ventaja**
 Desventaja
 Información

- Ejemplo:

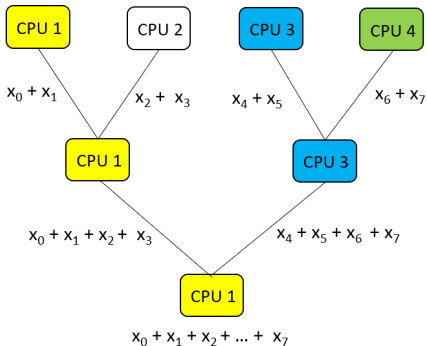


Figura 2: Suma de 8 números con 4 procesadores.

- Ejemplo:

- Ejemplo:

- ▶ Algoritmo secuencial:

```
1: función Suma(A, n)
2: if n = 1 then Return A[1]
3: else
4:   x = Suma(A [1 ...  $\frac{n}{2}$ ])
5:   y = Suma(A [ $\frac{n}{2} + 1$  ... n])
6:   Return {x + y}
7: end if
```

▷ Suma secuencial de n números

- Ejemplo:

- ▶ Algoritmo secuencial:

```
1: función Suma(A, n)
2: if n = 1 then Return A[1]
3: else
4:   x = Suma(A [1 ...  $\frac{n}{2}$ ])
5:   y = Suma(A [ $\frac{n}{2} + 1$  ... n])
6:   Return {x + y}
7: end if
```

▷ Suma secuencial de n números

- ▶ Ecuación recursiva: $T(n) = 2T(\frac{n}{2}) + 1$

- Ejemplo:

- Ejemplo:

- ▶ Algoritmo paralelo:

- 1: función PSuma($A[1 \dots n]$)

- 2: **if** $n = 1$ **then** Return $A[1]$

- 3: **end if**

- 4: En paralelo hacer

- 5: $x = \text{PSuma}(A[1 \dots \frac{n}{2}])$

- 6: $y = \text{PSuma}(A[\frac{n}{2} + 1 \dots n])$

- 7: Return $\{x + y\}$

- ▶ Suma paralela de n números

- ▶ 5: y 6: se ejecuta sobre otro procesador

- **Ejemplo:**

- ▶ **Algoritmo paralelo:**

- 1: función PSuma($A[1 \dots n]$)

- 2: **if** $n = 1$ **then** Return $A[1]$

- 3: **end if**

- 4: En paralelo hacer

- 5: $x = \text{PSuma}(A[1 \dots \frac{n}{2}])$

- 6: $y = \text{PSuma}(A[\frac{n}{2} + 1 \dots n])$

- 7: Return $\{x + y\}$

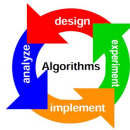
- ▶ Suma paralela de n números

- ▶ 5: y 6: se ejecuta sobre otro procesador

- ▶ **Ecuación recursiva:**

- Suponiendo que se tiene procesadores ($p \geq n$) para ejecutar cada par de llamadas en paralelo, tenemos la ecuación: $T(n) = T(\frac{n}{2}) + 1$

Algoritmos dividir para vencer



- 1 Dividir para vencer (conquistar)
- 2 Aplicaciones de algoritmos basados en Dividir para vencer
- 3 Problema del par mas cercano
- 4 Valor máximo y mínimo de un arreglo

Aplicaciones de algoritmos basados en Dividir para vencer

- Medicina:

Aplicaciones de algoritmos basados en Dividir para vencer

- **Medicina:**

- ▶ **Diagnóstico por imágenes (mamografías, resonancias, TAC):** Las imágenes médicas se dividen en regiones (e.g. por quadtree o segmentación) para detectar anomalías localmente.

Aplicaciones de algoritmos basados en Dividir para vencer

- **Medicina:**

- ▶ **Diagnóstico por imágenes (mamografías, resonancias, TAC):** Las imágenes médicas se dividen en regiones (e.g. por quadtree o segmentación) para detectar anomalías localmente.
- ▶ **Genómica y secuenciación del ADN:** Para alinear grandes cadenas de ADN, se dividen en fragmentos más cortos.

- Medicina: (cont.)

- **Medicina:** (cont.)

- ▶ **Clasificación de enfermedades mediante ML¹** : Los datos de pacientes (síntomas, exámenes) se dividen por grupo de órganos, sistemas o tipo de enfermedad, para ellos los modelos locales hacen clasificaciones por grupo, y luego un modelo combina las decisiones.

¹Machine Learning

- **Medicina:** (cont.)

- ▶ **Clasificación de enfermedades mediante ML¹** : Los datos de pacientes (síntomas, exámenes) se dividen por grupo de órganos, sistemas o tipo de enfermedad, para ellos los modelos locales hacen clasificaciones por grupo, y luego un modelo combina las decisiones.
- ▶ **Optimización de recursos hospitalarios:** El problema de asignación de camas, quirófanos o personal se divide por especialidades o turnos, para ello cada subproblema se resuelve de forma independiente, y luego se integran soluciones.

¹Machine Learning

- Logística:

- **Logística:**

- ▶ **Planeamiento de rutas a través del VRP:** Se divide la red logística (clientes, depósitos) en subzonas o rutas parciales. Luego cada subzona se optimiza por separado, luego se combinan para formar una solución global.

Ejemplo: Usado en Amazon, FedEx, DHL para entregas eficientes en grandes ciudades.

- **Logística:**

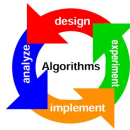
- ▶ **Planeamiento de rutas a través del VRP:** Se divide la red logística (clientes, depósitos) en subzonas o rutas parciales. Luego cada subzona se optimiza por separado, luego se combinan para formar una solución global.

Ejemplo: Usado en Amazon, FedEx, DHL para entregas eficientes en grandes ciudades.

- ▶ **Descomposición de redes de transporte.** Se divide una red de transporte global en sectores logísticos; luego cada uno se optimiza con restricciones locales: capacidad, tráfico, tiempo.

Ejemplo: Cadenas de frío (vacunas) usan este enfoque para garantizar cumplimiento de ventanas de tiempo por zonas.

Algoritmos dividir para vencer



- 1 Dividir para vencer (conquistar)
- 2 Aplicaciones de algoritmos basados en Dividir para vencer
- 3 Problema del par mas cercano
- 4 Valor máximo y mínimo de un arreglo

Problema del par mas cercano

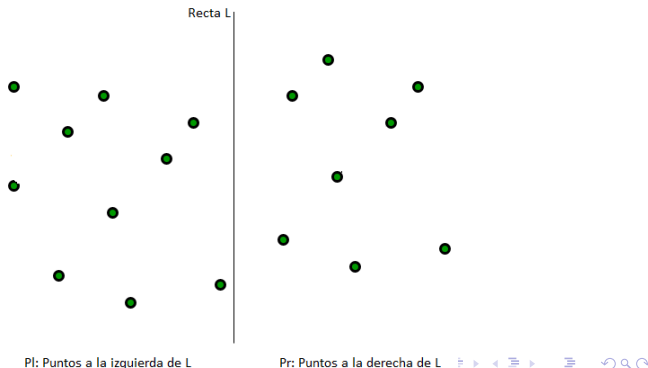
- **Datos:** Conjunto de distintos puntos: P en $\mathbb{R}^2$, ordenados en función de la abcisa x .

Problema del par mas cercano

- **Datos:** Conjunto de distintos puntos: P en $\mathbb{R}^2$, ordenados en función de la abscisa x .

- ▶ **Dividir si $n > 3$:** Dos subconjuntos de puntos $P_l = \lceil n/2 \rceil$ $P_r = \lfloor n/2 \rfloor$

trazando una recta vertical a través de la mediana m de sus x coordenadas, de modo que $\lceil n/2 \rceil$ puntos esten a la izquierda o sobre la recta vertical; y $\lfloor n/2 \rfloor$ puntos que esten a la derecha o sobre la recta vertical.



- Datos (cont.)

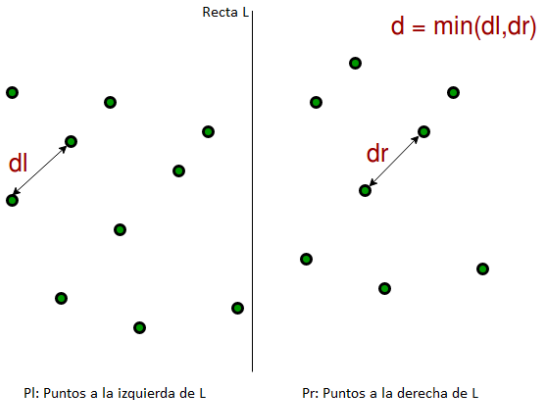
- Datos (cont.)

- ▶ Solucionar: El problema recursivamente para P_l y P_r .

- Datos (cont.)

- ▶ Solucionar: El problema recursivamente para P_l y P_r .

- ★ Sean d_l y d_r la menor distancia entre pares de puntos en P_l y P_r . Sea además $d = \min \{d_l, d_r\}$.



- Datos (cont.)

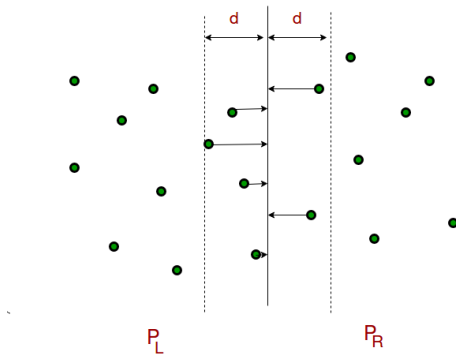
- Datos (cont.)
 - ▶ Solucionar: (cont.)

- Datos (cont.)
 - ▶ Solucionar: (cont.)

● Datos (cont.)

► Solucionar: (cont.)

- ★ d no necesariamente es la menor distancia entre todos los pares de puntos, pues los puntos de un par cercano pueden estar en lados opuestos de la recta. Por lo tanto, para combinar las soluciones de los subproblemas, examinar tales puntos. Es posible limitar la atención a los puntos que están dentro de la vertical simétrica de ancho $2d$ alrededor de la recta separador (líneas punteadas), ya que la distancia entre cualquier otro par de puntos es al menos d .



- Tarea:

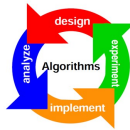
- Tarea:

- ▶ Investigar un pseudocódigo basado en la estrategia dividir para vencer aplicado al problema del par mas próximo.

- Tarea:

- ▶ Investigar un pseudocódigo basado en la estrategia dividir para vencer aplicado al problema del par mas próximo.
- ▶ Ecuación recursiva: $T(n) = T(\lceil \frac{n}{2} \rceil) + T(\lfloor \frac{n}{2} \rfloor) + n$

Algoritmos dividir para vencer



- 1 Dividir para vencer (conquistar)
- 2 Aplicaciones de algoritmos basados en Dividir para vencer
- 3 Problema del par mas cercano
- 4 Valor máximo y mínimo de un arreglo

Valor máximo y mínimo de un arreglo

- **Problema:** Hallar el mayor y menor elemento en un vector $A[1 \dots n], \forall n \geq 1$.

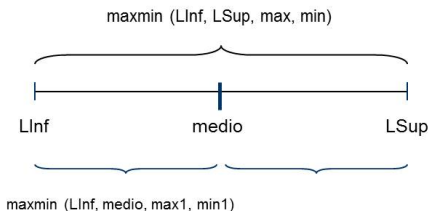


Figura 3: Proceso para el algoritmo maxmin.

• Algoritmo:

```
1: maxmin (LInf, LSup, max, min)
2: if (LSup - LInf)  $\leq$  1 then
3:   if (A[LInf]) < A[LSup] then
4:     max := A[LSup]
5:     min := A[LInf]
6:   else
7:     max := A[LInf]
8:     min := A[LSup]
9:   end if
10: else
11:   medio :=  $\left\lfloor \frac{LInf + LSup}{2} \right\rfloor$ 
12:   maxmin (LInf, medio, max1, min1)
13:   maxmin (medio + 1, LSup, max2, min2)
14:   if max1 > max2 then
15:     max := max1
16:   else
17:     max := max2
18:   end if
19:   if min1 < min2 then
20:     min := min1
21:   else
22:     min := min2
```